

Agentic RAG System — Project Write-Up

Introduction

The goal of this project was to develop an Agentic Retrieval-Augmented Generation (RAG) system capable of ingesting documents, generating embeddings, storing them in a vector database, retrieving relevant information semantically, and generating grounded responses using a Large Language Model (LLM).

The system was designed to minimize hallucinations and answer questions strictly based on the ingested documents. The final implementation supports document ingestion, semantic retrieval, conversational querying, source attribution, and a web-based interface.

System Architecture

User Query → Retriever → ChromaDB Vector Store → Relevant Context Chunks → Prompt Construction → LLM (Llama3 via Ollama) → Grounded Response

The pipeline consists of the following stages:

1. Document Ingestion
 - PDF, TXT, and CSV documents are loaded using LangChain document loaders.
2. Text Chunking
 - Documents are split into smaller chunks using RecursiveCharacterTextSplitter.
 - Chunk overlap is used to preserve context continuity.
3. Embedding Generation
 - Text chunks are converted into dense vector embeddings using sentence-transformers.
4. Vector Database Storage
 - Embeddings are stored persistently in ChromaDB for semantic similarity retrieval.
5. Semantic Retrieval
 - User queries are converted into embeddings and compared using cosine similarity search.
6. Grounded Response Generation
 - Retrieved context is injected into the LLM prompt.
 - The LLM generates answers strictly based on retrieved content.

Architecture Decisions

Why ChromaDB?

- Lightweight and easy to integrate with LangChain
- Supports persistent local vector storage
- Efficient semantic similarity search
- Ideal for local development and experimentation

Why sentence-transformers?

Model Used: all-MiniLM-L6-v2

Reasons:

- Lightweight and fast
- Local execution without API dependency
- Good semantic embedding quality
- Suitable for CPU-based systems

Why Ollama + Llama3?

- Fully local execution
- No API cost
- Better privacy
- Offline capability
- Easy deployment and experimentation

Hallucination Prevention Strategy

1. Strict grounded prompting
2. Retrieval score thresholding
3. Explicit fallback responses
4. Low temperature generation
5. Source attribution and chunk transparency

Fallback Response Example:

"I could not find relevant information in the provided documents."

Features Implemented

- Multi-format document ingestion
- Semantic chunking
- Embedding generation
- ChromaDB vector storage
- Semantic retrieval
- Conversational querying
- Streamlit web interface
- CLI support
- Conversational memory
- Tool orchestration
- Source attribution
- Retrieval transparency

Limitations

1. Large documents may exceed context window limits.
2. Retrieval quality depends heavily on chunking strategy.
3. No multimodal support for images or tables.
4. Retrieval may occasionally miss relevant chunks for ambiguous queries.

Future Improvements

- Hybrid retrieval (BM25 + vector search)
- Re-ranking models
- Multi-query retrieval
- Cloud deployment
- Streaming responses
- Authentication system
- GPU acceleration
- Agentic web-search integration

Conclusion

The final system successfully demonstrates the core principles of Retrieval-Augmented Generation (RAG), including semantic retrieval, vector databases, grounded prompting, and hallucination mitigation.

The project demonstrates practical AI engineering skills involving retrieval systems, semantic search, vector databases, local LLM orchestration, explainable AI interfaces, and modular software architecture.